MODULE 22 · DOMAIN 3 - DEPLOYMENT

CI/CD – CodePipeline, CodeBuild and CodeArtifact

Stages and actions, the buildspec phases, and where artifacts actually live between them.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 22 of 34

admin@sthithapragna.com

A pipeline moves **artifacts** between stages. Almost every failure is either a missing artifact or a missing permission.

The pipeline model

- A pipeline has **stages**, executed in order. Each stage contains one or more **actions**.
- Every pipeline needs at least **two stages**, and the first must be a source stage.
- A stage succeeds only if every action in it succeeds. A failure stops the pipeline there.
- Actions declare **input artifacts** and **output artifacts** by name.
- An action can only consume an artifact produced by an **earlier** stage.

> ON THE EXAM
>
> The naming is what links stages together. An action asking for an artifact name that nothing produced fails validation before the pipeline even runs.

The action categories

CATEGORY	TYPICAL PROVIDERS
Source	CodeCommit, S3, ECR, GitHub and others via CodeConnections
Build	CodeBuild, Jenkins
Test	CodeBuild, third-party testing tools
Deploy	CodeDeploy, CloudFormation, ECS, S3, Elastic Beanstalk, Service Catalog
Approval	Manual approval only
Invoke	Lambda, Step Functions

ON THE EXAM

The Invoke category is the useful escape hatch: a Lambda action can do anything the pipeline has no built-in provider for.

Artifacts

- Artifacts are zip files stored in an **S3 artifact bucket** for the pipeline.
- The source action produces the first one. Each later action can add more.
- **CodeBuild only uploads what its buildspec artifacts section lists.** Nothing else survives.
- The bucket is encrypted, and the pipeline role needs read and write access to it.
- Cross-region actions need an artifact bucket in **each** Region involved.

> ON THE EXAM
>
> The third point causes the most common CI/CD failure there is: a build that passes, and a deploy stage that finds nothing to deploy.

Ordering within a stage

- Actions in a stage with the **same runOrder** execute in **parallel**.
- Different runOrder values execute in sequence, lowest first.
- So parallel test suites are one stage with several actions sharing a runOrder.
- Up to 50 actions per stage, and 500 per pipeline.
- A parallel action cannot consume an artifact produced by another action at the same runOrder.

> ON THE EXAM
>
> runOrder is the only concurrency control inside a stage. Three test suites at runOrder 1 run together; a deploy at runOrder 2 waits for all three.

Manual approval

- An approval action **pauses the pipeline** until someone approves or rejects it.
- It can publish to an **SNS topic**, so approvers are notified rather than watching the console.
- A review URL and comments can be attached, giving the approver context.
- It times out after **7 days** and the pipeline fails if nobody responds.
- Approvers need **codepipeline:PutApprovalResult** on that stage.

> ON THE EXAM
>
> Approval actions produce no artifacts and consume none. They exist purely to stop the pipeline, which is why they sit in their own stage.

Watching a pipeline

- CodePipeline emits **EventBridge events** for every stage and action state change.
- So a failed stage can trigger a Lambda function, an SNS topic or a chat notification.
- **Notification rules** are the built-in route, sending to SNS or AWS Chatbot.
- EventBridge is also how one pipeline **starts another**, for chained deployments.
- Nobody watches a console. If a failure has no notification, it is discovered by a user.

> ON THE EXAM
>
> The same EventBridge pattern from module 12 applies here: the pipeline publishes, and rules decide who cares.

Source providers

- **CodeConnections**, formerly CodeStar Connections, links GitHub, GitLab and Bitbucket.
- A connection is authorised once and reused across pipelines. No personal access tokens per repository.
- **CodeCommit** and **S3** sources are native and need no connection.
- **Polling for source changes is legacy.** EventBridge or webhook triggers are faster and cheaper.
- An **ECR** source starts a pipeline when a new container image is pushed.

> ON THE EXAM
>
> Polling checks periodically and adds minutes of delay. Event-driven triggers start the pipeline within seconds, which is why AWS disables polling on new pipelines.

Currency note: pipeline type V2

- **The examinable model** is the V1 pipeline: stages, actions and artifacts.
- **V2** is now the default and adds four things V1 lacks.
- **Triggers with filters** on branches, file paths, pull requests and git tags.
- **Pipeline-level variables**, passed in at execution time.
- **Stage conditions** and **stage rollback**, plus **parallel** and **queued** execution modes.

> ON THE EXAM
>
> V2 is priced differently: V1 is a flat monthly fee per pipeline, V2 is per action-minute. That matters for pipelines that run rarely.

Execution modes and stage conditions

- **SUPERSEDED** is the default. A newer execution overtakes and cancels an older one.
- **QUEUED** runs executions one at a time, in order, with none discarded.
- **PARALLEL** runs executions independently, with no ordering at all.
- **Stage conditions** evaluate rules on entry, on success or on failure.
- A failing condition can **Skip** the stage, **Fail** it, or trigger an automatic **Rollback**.

ON THE EXAM

SUPERSEDED explains a real surprise: push twice quickly and the first execution is cancelled mid-flight. QUEUED is the fix when every commit must deploy.

Variables and namespaces

- An action can declare a **namespace**, exposing its output variables to later actions.
- Referenced as **#{Namespace.VariableName}** in a later action's configuration.
- A source action publishes **CommitId**, **BranchName** and similar.
- CodeBuild exports variables listed in its buildspec **exported-variables** section.
- Variables flow **forward only**, exactly like artifacts.

> ON THE EXAM
>
> This is how a commit ID reaches a deploy action without being written into an artifact. Small values travel as variables; files travel as artifacts.

AWS CodeBuild

- Runs builds in a managed container. No build servers to maintain, billed per build minute.
- A **build project** defines the source, environment, service role and buildspec.
- Environments are AWS-provided images, or your own image from ECR or Docker Hub.
- It can run inside a **VPC** to reach private resources, at the cost of slower start-up.
- The **service role** is CodeBuild's own identity. Missing permissions here are a frequent failure.

> ON THE EXAM
>
> CodeBuild is stateless: each build starts from a fresh container. Anything you want to persist must be an artifact, a cache, or written somewhere yourself.

buildspec anatomy

SECTION	WHAT IT DOES
`version`	0.2 for anything current
`env`	Variables, plus parameter-store and secrets-manager blocks
`phases`	install, pre_build, build, post_build
`artifacts`	Which files are uploaded as output. Nothing else survives
`cache`	Paths preserved between builds
`reports`	Test and coverage report groups

ON THE EXAM

By default it must be named buildspec.yml at the repository root, though the project can point elsewhere. YAML indentation errors are the usual first failure.

The four phases

- **install** for runtime versions and build tooling. **pre_build** for setup such as logging in to ECR.
- **build** for compiling and testing. **post_build** for packaging and pushing images.
- If any command fails, the phase fails and the build fails.
- **post_build runs even if build failed**, which is deliberate.
- So post_build is where you publish test reports that explain the failure.

> ON THE EXAM
>
> That post_build behaviour is examinable and useful. It runs on failure so that diagnostic output still gets published, but the build is still marked failed.

Environment variables in a build

- **env/variables** holds plain values. Visible to anyone who can read the buildspec.
- **env/parameter-store** resolves values from Parameter Store at build time.
- **env/secrets-manager** resolves values from Secrets Manager at build time.
- The project itself can set variables, and a pipeline action can override them.
- CodeBuild masks values it resolved itself in logs. It cannot mask what your script echoes.

> ON THE EXAM
>
> Module 19 made the same point from the secrets side. A resolved secret echoed by your own script reaches the build log in plain text.

Artifacts and caching

- The **artifacts** section lists files to upload, with **files**, **base-directory** and **discard-paths**.
- **secondary-artifacts** produces several named outputs from one build.
- **S3 caching** stores paths in a bucket between builds. Good for dependencies.
- **Local caching** keeps a layer on the build host. Faster, and only helps consecutive builds.
- Caching node_modules or a Maven repository is usually the biggest single speed win.

> ON THE EXAM
>
> A cache is an optimisation, never a dependency. A build that only works with a warm cache will fail the first time it runs on a fresh host.

Test reports

- A **reports** section points at test output files and a report group.
- Formats: JUnit XML, Cucumber JSON, TestNG, Visual Studio TRX and others.
- The console then shows pass and fail counts, durations and trends over time.
- **Code coverage reports** are supported in JaCoCo, Cobertura, SimpleCov and Clover.
- Reports are produced even when the build fails, provided the phase runs.

> ON THE EXAM
>
> Without reports, a failed test is a wall of console output. With them, the console shows which test failed and how the failure rate is trending.

Debugging a build

- Build logs go to **CloudWatch Logs**, and optionally to S3.
- **Session Manager** can attach to a running build, pausing it with codebuild-breakpoint.
- That gives a shell inside the actual build container, which is far faster than guessing.
- **Local builds** run the same image on your machine using the CodeBuild agent.
- **Most build failures are permissions or a missing file**, not the build tool itself.

> ON THE EXAM
>
> Session Manager access to a paused build is the underused feature here. It turns 'why does this only fail in CI' into a question you can answer directly.

Build performance and cost

- Billing is per build minute, by **compute type**. A larger instance costs more per minute.
- A bigger instance is often **cheaper overall**, because the build finishes proportionally faster.
- **Dependency caching** usually saves more than any compute change.
- **Batch builds** run several configurations in parallel, such as a build matrix.
- A custom image with tooling pre-installed removes install-phase time from every build.

> ON THE EXAM
>
> The counterintuitive one is compute size. Doubling the instance rarely doubles the bill, because minutes fall at the same time as the per-minute rate rises.

AWS CodeArtifact

- Managed artifact repository for **npm, PyPI, Maven, NuGet, Ruby, Swift** and generic packages.
- A **domain** contains **repositories**. Storage is deduplicated across the domain.
- Authentication uses an **IAM-derived token** from the login command. No static registry credentials.
- Repositories have resource policies, so they can be shared across accounts.
- Assets are encrypted with KMS, and every fetch is recorded in CloudTrail.

> ON THE EXAM
>
> The IAM integration is the real difference from running your own registry: package access is governed by the same policies as everything else in the account.

Upstreams and external connections

- An **external connection** links a repository to a public registry such as npmjs or PyPI.
- A package requested but not present is fetched from upstream and then **cached permanently**.
- So a later removal from the public registry does not break your builds.
- **Upstream repositories** chain CodeArtifact repositories together.
- The usual shape: a team repository upstream of a shared repository upstream of the public registry.

> ON THE EXAM
>
> The permanent caching is the resilience argument. A package deleted from a public registry stays available to you, because you already have a copy.

Package origin controls

- **The attack:** publish a public package with the same name as an internal one, at a higher version.
- A naive resolver prefers the higher version and installs the attacker's code.
- **Package origin controls** decide whether a package may be published, fetched from upstream, or both.
- Setting upstream to BLOCK for an internal package name means the public version is never pulled.
- CodeArtifact applies this automatically for packages first published internally.

> ON THE EXAM
>
> This is a real supply-chain attack with a specific named defence. If a scenario describes internal package names appearing publicly, this is the answer.

Currency note: CodeCommit reopened

- **The examinable position:** CodeCommit is a source provider like any other.
- AWS **closed it to new customers in June 2024**, and much writing still treats it as dead.
- It was **reopened to general availability in November 2025**, with an active roadmap.
- So it is a live, actively developed service again, not a legacy one.
- Answer normally on the exam. CodeCommit remains a valid source action.

> ON THE EXAM
>
> Worth knowing because a lot of 2024 and 2025 material confidently states CodeCommit is being retired. That was true, and it is not true now.

Cross-account and cross-region

- A pipeline in a tooling account can deploy into **separate dev, staging and production accounts**.
- Each action then needs a **role in the target account** that the pipeline assumes.
- The artifact bucket must use a **customer managed KMS key**, shared with the target accounts.
- **The default AWS managed key cannot be shared**, which is the usual first failure.
- Cross-region actions need an artifact bucket in each Region, which CodePipeline can create.

> ON THE EXAM
>
> Module 18's rule again: cross-account access to an encrypted artifact needs the key policy to allow it, and the default key never will.

Diagnosing a pipeline failure

SYMPTOM	USUAL CAUSE
Deploy stage finds no files	The buildspec artifacts section did not list them
Build fails immediately, no phase output	buildspec.yml missing, misnamed, or invalid YAML
Build cannot reach a private database	The project is not configured for the VPC
Cross-account deploy fails on decrypt	Artifact bucket uses the default KMS key
Pipeline never starts on a push	Polling disabled and no trigger configured
An execution disappears mid-run	SUPERSEDED mode. A newer execution overtook it
Access denied writing artifacts	The CodeBuild service role lacks S3 permissions

ON THE EXAM

Notice how many are configuration rather than code. The pipeline is usually working exactly as configured, which is the problem.

Ask first: is this about **what moved**, or about **who was allowed**?

A pipeline has a source stage, a CodeBuild stage and a CodeDeploy stage. The build stage reports success, tests pass and the console shows green. The deploy stage then fails, reporting that the input artifact contains no deployable files.

The build writes its compiled output to a dist directory and the appspec file to the repository root.

What is the most likely cause?

A The CodeDeploy service role lacks permission to read the artifact bucket

B The build and deploy stages are in different Regions

C The build stage needs a longer timeout so files finish writing

D The buildspec artifacts section does not list the dist directory and the appspec file

◆ PAUSE AND DECIDE

What is the most likely cause?

A CodeDeploy role permissions

That would produce an access denied error, not a report that the artifact contains nothing. The artifact was read successfully; it was empty.

B Different Regions

A cross-Region artifact problem fails on retrieval with a bucket error, and the pipeline would have failed validation earlier.

C Build timeout

A timeout fails the build. This build succeeded, and CodeBuild does not upload partially written files.

D Artifacts section incomplete

CodeBuild uploads only what the artifacts section lists. A green build that produces an empty artifact is this, essentially every time.

THE SIGNAL

'Build green, deploy finds nothing' is the single most recognisable CI/CD symptom. The build did its job; it just was not asked to keep anything.

A pipeline deploys to staging automatically. Compliance requires that a named release manager explicitly authorises the production deployment, and that they are notified rather than having to watch the console.

If nobody responds, the deployment must not proceed. The team does not want to build a custom approval application.

How should this be implemented?

A A Lambda invoke action that polls a DynamoDB approvals table until a record appears

B A manual approval action in its own stage, publishing to an SNS topic the release manager subscribes to

C Disable the production stage and enable it manually when a release is authorised

D A stage condition that checks a pipeline variable set by the release manager

◆ PAUSE AND DECIDE

How should this be implemented?

A Lambda polling DynamoDB

Rebuilds manual approval by hand, including the notification, the timeout and the audit trail. It is the custom application the team ruled out.

B Manual approval with SNS

Purpose-built: the pipeline pauses, SNS notifies the approver, PutApprovalResult permission names who may approve, and it fails after seven days.

C Disable the stage

Not an approval, and it leaves no record of who authorised the release, which is what compliance actually asked for.

D Stage condition on a variable

Conditions evaluate automatically against rules. There is no human in the loop, and no notification.

THE SIGNAL

The compliance angle is what makes B right rather than merely convenient: the approval is recorded, attributed to a principal, and governed by an IAM permission.

> A CodeBuild project must authenticate to a third-party API during the build. The API key currently sits in the buildspec's env/variables block, and a security review has flagged that anyone with repository access can read it.
>
> The key rotates every 30 days. The build log must not contain it.

How should the key be supplied?

A Store it in Secrets Manager and reference it from the buildspec's env/secrets-manager block

B Set it as a plaintext environment variable on the build project instead of in the buildspec

C Encrypt the key with KMS and paste the ciphertext into env/variables

D Pass it as a pipeline-level variable at execution time

◆ PAUSE AND DECIDE

How should the key be supplied?

A secrets-manager block

The buildspec holds only a reference, the value resolves at build time, rotation is handled by Secrets Manager, and CodeBuild masks values it resolved itself.

B Plaintext project variable

Moves the secret out of the repository and into the project configuration, readable by anyone who can describe the project. Still plaintext.

C Ciphertext in the buildspec

Requires decrypting in the build, so your script handles the plaintext and can echo it. It also breaks on every rotation.

D Pipeline-level variable

Someone must supply the value at execution time, so it ends up in a console field and in the execution history.

THE SIGNAL

Three requirements: not in the repository, rotates, and not in the log. Only the secrets-manager block satisfies all three without extra code.

> A company publishes internal npm packages under names like @acme/auth-client. A security audit warns that an attacker could publish a package with the same name to the public npm registry at a higher version, and builds might resolve to it.
>
> Builds must still be able to install genuine public dependencies from npm.

What should the company implement?

A Pin every dependency to an exact version in package-lock.json

B Remove the external connection to npm and vendor all public packages manually

C CodeArtifact with an external connection to npm, and package origin controls blocking upstream versions of internal package names

D Run npm audit as a build phase and fail the build on any finding

◆ PAUSE AND DECIDE

What should the company implement?

A Pin exact versions

Helps, and breaks the moment anyone adds or upgrades a dependency. It is a discipline rather than a control, and one mistake reopens the hole.

B Vendor everything manually

Eliminates the risk by eliminating the convenience. It also contradicts the requirement to install genuine public dependencies.

C CodeArtifact origin controls

The named defence for dependency confusion. Internal package names can never be fetched from upstream, while genuine public packages still resolve normally.

D npm audit in the build

Reports known vulnerabilities in real packages. A malicious package impersonating an internal name is not a known vulnerability.

THE SIGNAL

This attack has a specific name, dependency confusion, and a specific AWS answer. Recognising the shape is what makes the question quick.

What to carry forward

- **Only what the buildspec artifacts section lists survives the build.** The commonest failure there is.
- **post_build runs even when build fails**, so diagnostics still get published.
- Secrets reach a build through **env/secrets-manager**, never through env/variables.
- Same runOrder means parallel. Manual approval pauses the pipeline and times out at 7 days.
- Cross-account pipelines need a **customer managed KMS key** on the artifact bucket.

> ON THE EXAM
>
> Next: CodeDeploy and deployment strategies - what the deploy stage actually does once it has the artifact.